

LECTURE 10

Responsive Design and Frameworks

Building one site that adapts to any screen — and the two major CSS frameworks that make it faster.

CSC336 Web Technologies

In This Lecture

- 1 Mobile-first design, the viewport meta tag, and fluid units
- 2 Media queries and breakpoints — every combination, plus responsive images
- 3 Component-based (Bootstrap) vs. utility-first (Tailwind) frameworks
- 4 Bootstrap's grid system, components, and utilities
- 5 Tailwind's utility-class structure
- 6 Extending either framework with your own custom classes

Mobile-First Design

- 1 Write base CSS for small screens FIRST, then add styling for larger screens with media queries
- 2 The opposite of "desktop-first," where you design big and cram it onto a phone afterward
- 3 Mobile traffic is the majority of web visits — adding complexity as space grows is easier than stripping it away

The Viewport Meta Tag

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

width=device-width uses the real device width; initial-scale=1.0 starts at 100% zoom.

Never Forget This Tag

WARNING

Without the viewport meta tag, none of your media queries will work correctly on a real phone — the browser will still pretend it has a much wider screen.

Units That Scale, Not Fixed Pixels

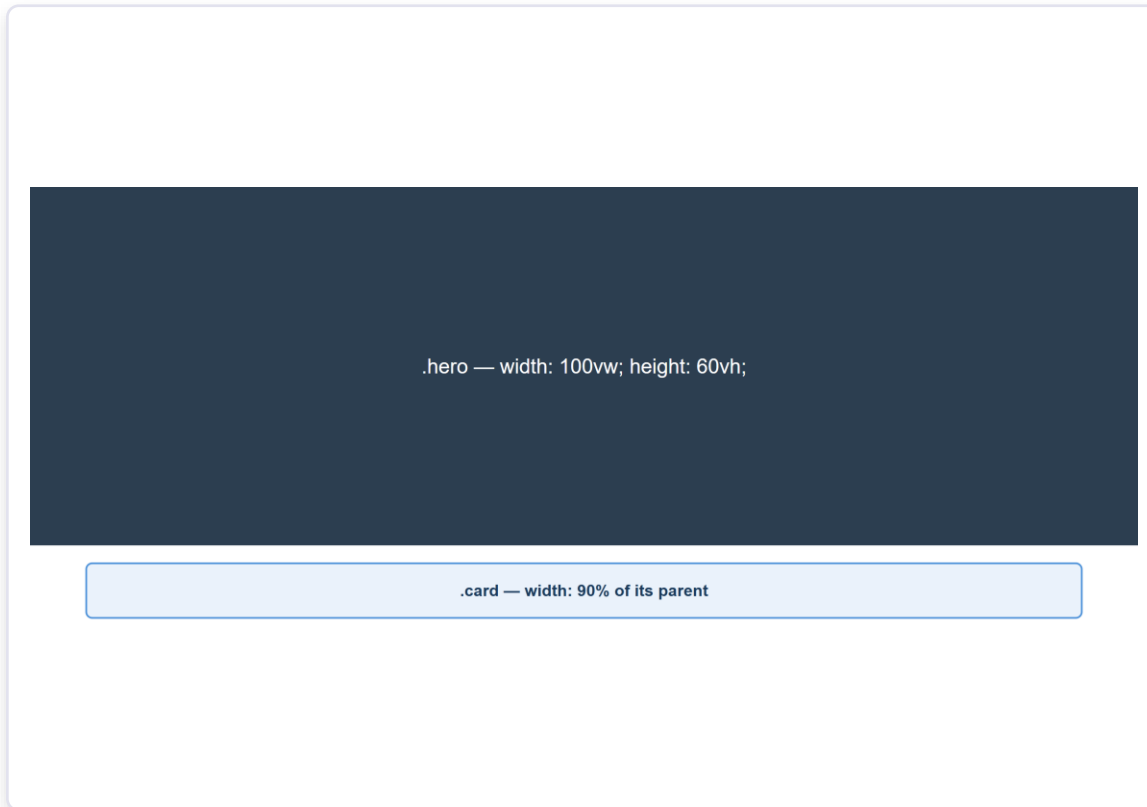
Unit	Relative To
%	The size of the parent element
rem	The root (<html>) element's font size
em	The current element's own font size
vw / vh	1% of the viewport's width / height

Fluid Units in Practice

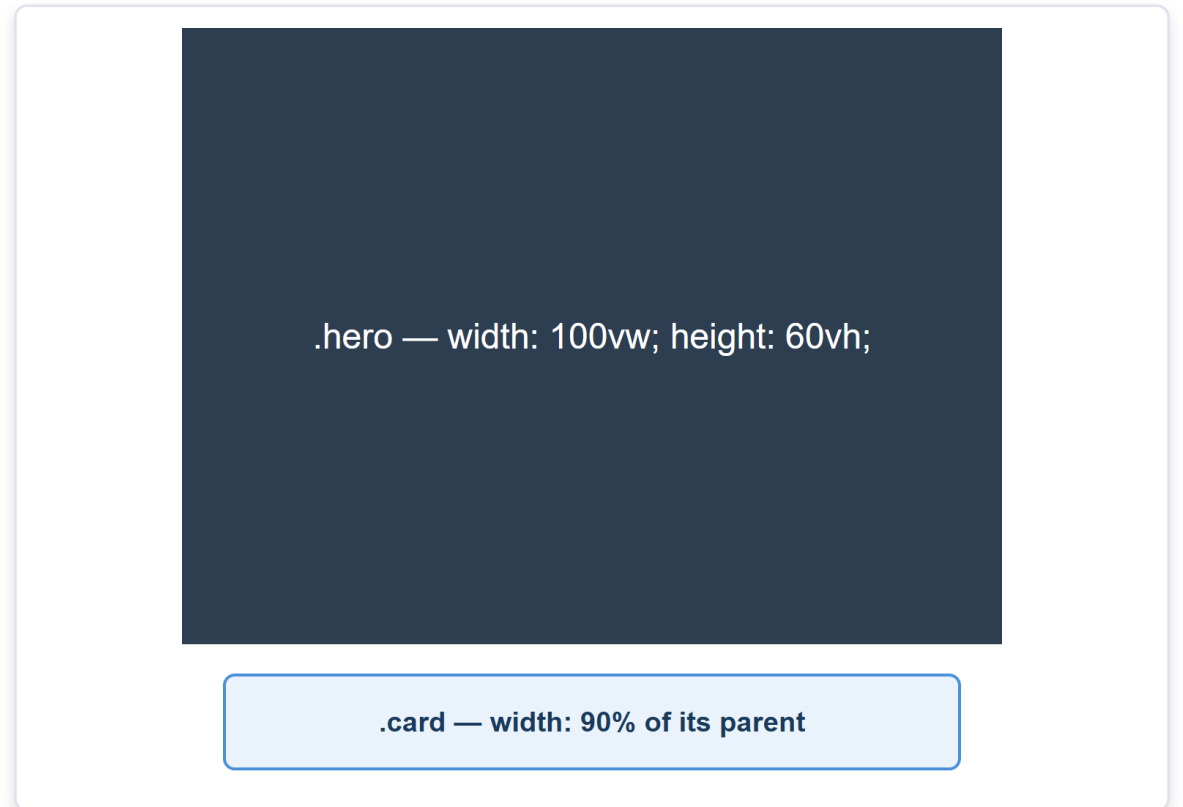
```
.hero {  
  width: 100vw;    /* always fills the full screen width */  
  height: 60vh;    /* always 60% of the visible screen height */  
}  
.card { width: 90%; } /* always 90% of its parent, whatever that is */
```

The Same Page, Two Widths

WIDE



NARROW



A Mobile-First Breakpoint

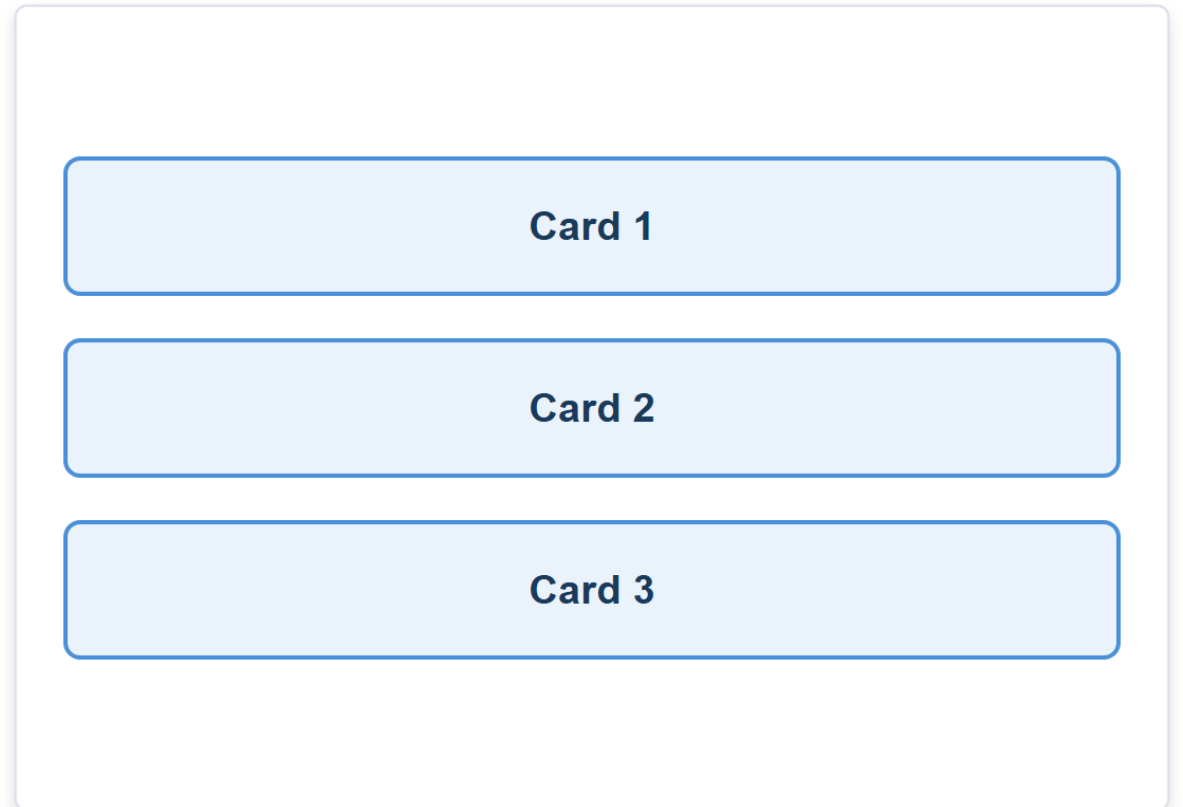
```
.container { display: flex; flex-direction: column; }  
  
@media (min-width: 768px) {  
  .container { flex-direction: row; }  
}
```

Three Cards, Two Widths

WIDE (ROW)



NARROW (COLUMN)



and, Comma, and not

Operator	Meaning
and	Combines conditions — ALL must be true
, (comma)	Works like OR — ANY one condition must be true
not	Inverts the entire media query (needs a media type)
orientation / prefers-color-scheme / print	Portrait vs. landscape / dark mode / printed page — same @media syntax

The One Rule That Matters

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

max-width: 100% keeps an image from ever growing wider than its container; height: auto preserves its aspect ratio.

The Same Image, Two Widths

WIDE

Container is 60% of the page width. The image's real file is 800×400px.



NARROW

Container is 60% of the page width. The image's real file is 800×400px.



Component-Based vs. Utility-First

BOOTSTRAP (COMPONENT-BASED)

- Ships pre-styled, ready-to-use components (.btn, .card, .navbar)
- Fast to get a decent-looking page up quickly
- Overriding built-in styles can fight the framework

TAILWIND (UTILITY-FIRST)

- Ships hundreds of tiny, single-purpose utility classes (flex, p-4)
- Nothing looks "designed" until you style it yourself
- Takes longer at first — many class names to learn

The 12-Column Grid

```
<div class="container">
  <div class="row">
    <div class="col-md-8">Main content</div>
    <div class="col-md-4">Sidebar</div>
  </div>
</div>
```

The Grid, Two Widths

WIDE (SIDE BY SIDE)

Main content (8 of 12 columns on medium screens+)

Sidebar (4 of 12 columns on medium screens+)

NARROW (STACKED)

Main content (8 of 12 columns on medium screens+)

Sidebar (4 of 12 columns on medium screens+)

Components, Rendered

Three lines of HTML, zero custom CSS.

```
<button class="btn btn-primary">Save Changes</button>

<div class="card"><div class="card-body">
  <h5 class="card-title">Card Title</h5>
</div></div>

<nav class="navbar navbar-expand-lg navbar-light bg-light">...</nav>
```

Save Changes

Card Title

Some quick example text for this card component.

Read more

MySite

Utilities, Rendered

```
<div class="d-flex justify-content-between p-3">  
  <!-- d-flex: display:flex; justify-content-between: space-between; p-3: padding -->  
</div>
```

Left item

Right item

A Fully Styled Button

```
<button class="bg-blue-600 hover:bg-blue-700 text-white font-semibold  
            px-4 py-2 rounded-lg shadow">  
  Save Changes  
</button>
```

The Button, Rendered



Save Changes

Responsive by Default

```
<div class="flex flex-col md:flex-row">  
  <!-- stacked by default, a row from md breakpoint up -->  
</div>
```

flex-col md:flex-row, Two Widths

WIDE (ROW)



NARROW (COLUMN)



Responsive Width

```
<div class="w-full lg:w-1/3">  
  <!-- full width by default, one-third width from the "lg" breakpoint up -->  
</div>
```

w-full lg:w-1/3, Two Widths

WIDE (1/3)

w-full by default, lg:w-1/3 from the "lg" breakpoint up

NARROW (FULL)

w-full by default, lg:w-1/3 from the "lg" breakpoint up

Bootstrap + Your Own Class

```
<button class="btn btn-primary my-cta-button">Get Started</button>
```

```
.my-cta-button { text-transform: uppercase; letter-spacing: 1px; }
```

Bootstrap + Custom, Rendered



GET STARTED

Tailwind + Your Own Class

```
<span class="text-blue-600 font-bold brand-underline">New!</span>
```

```
.brand-underline { text-decoration: underline wavy; text-underline-offset: 4px; }
```

Tailwind + Custom, Rendered

New!



KEY TAKEAWAYS

Lecture 10 in Six Points

- 1 Mobile-first: base styles for small screens, then add complexity with min-width media queries.
- 2 The viewport meta tag is required for media queries to work correctly on real phones.
- 3 Fluid units (% , rem, vw, vh) scale relative to something else, unlike fixed px.
- 4 Media queries combine with and (all true), comma (any true), and not (inverts) — and test more than width.
- 5 max-width: 100%; height: auto; keeps images from overflowing their container.
- 6 Bootstrap gives finished components; Tailwind gives composable utilities — both expect your own custom CSS on top.